

Introduction to Transactions

1. Database Transaction

A **database transaction** is a group of database operations that the database treats as one logical unit of work.

Consider a bank transfer. The initial state is:

```
Account A = ₹10,000
Account B = ₹5,000
```

To transfer ₹1,000 from Account A to Account B, the application executes two updates:

```
UPDATE account
SET balance = 9000
WHERE id = 1;

UPDATE account
SET balance = 6000
WHERE id = 2;
```

Technically, these are two separate SQL statements. From the business perspective, however, they represent one operation:

```
Transfer ₹1,000 from Account A to Account B
```

The complete operation should move the database from one valid state to another:

Before the transaction:

```
Account A = ₹10,000
Account B = ₹5,000
Total     = ₹15,000
```

After the transaction:

Account A = ₹9,000
Account B = ₹6,000
Total = ₹15,000

A transaction is therefore not defined by the number of SQL statements. It is defined by the business operation whose steps must succeed or fail together.

| A transaction is a unit of work that must be completed fully or not applied at all.

2. ACID Properties

ACID describes the guarantees required for reliable transaction processing:

- **A — Atomicity**
- **C — Consistency**
- **I — Isolation**
- **D — Durability**

2.1 Atomicity

Atomicity means that all operations inside a transaction succeed together or all of them are discarded together.

In the transfer example:

Debit Account A
Credit Account B

Both operations must succeed. If Account A is debited but Account B cannot be credited, the debit must also be rolled back.

Both balances change, or neither balance changes.

Without atomicity, a failure between the two updates could cause money to disappear.

2.2 Consistency

Consistency means that a transaction should move the database from one valid state to another valid state.

Suppose the business rule is:

An account balance cannot be negative.

The current balance is:

Account A = ₹500

The application attempts to transfer ₹1,000. If the operation leaves the account in this state:

Account A = -₹500

the business rule has been violated and the database is now in an invalid state.

Consistency depends on correctly defined rules and their enforcement through application logic, database constraints, or both. A transaction does not invent these rules; it ensures that the committed result does not contain only part of the intended operation.

In the transfer example, consistency can include rules such as:

- Account balances must remain valid.
- The total amount of money must not change because of an internal transfer.
- The sender and receiver must both exist.

2.3 Isolation

Isolation deals with transactions executing concurrently.

Suppose Account A contains ₹10,000 and two transactions execute at approximately the same time:

```
Transaction T1: Withdraw ₹1,000
Transaction T2: Withdraw ₹8,000
```

Without sufficient isolation, both transactions may read the same initial balance:

```
T1 reads ₹10,000
T2 reads ₹10,000
```

They then calculate and write different values:

```
T1 writes ₹9,000
T2 writes ₹2,000
```

Depending on the execution order, one update may overwrite the other. The final balance may fail to reflect both withdrawals correctly.

Isolation answers the following question:

What is one transaction allowed to observe while another transaction is still executing?

Isolation does not necessarily mean that only one transaction can execute at a time. Databases allow concurrency, but they control how concurrent operations interact.

2.4 Durability

Durability means that once a transaction has committed successfully, its changes should survive later failures.

Suppose the database confirms:

```
Transfer completed successfully.
```

Immediately afterward:

- The application crashes.
- The database process restarts.
- The server loses power.

After recovery, the committed transfer should still exist.

Commit is a promise that the database has accepted the transaction as a permanent result.

3. ACID as One Combined Guarantee

The four ACID properties are not unrelated definitions. Together, they describe whether a database can safely execute a business operation despite failures and concurrency.

For the bank transfer:

Property	Guarantee
Atomicity	Both balances change, or neither changes.
Consistency	Money is not created or destroyed, and all business rules remain valid.
Isolation	Concurrent operations do not corrupt the transfer.
Durability	Once committed, the transfer survives subsequent failures.

Together, ACID answers one central question:

Can the database safely move from one valid state to another despite failures and concurrent activity?

4. Transaction Boundary

A **transaction boundary** defines where a transaction begins and where it ends.

In Spring, a service method can represent the boundary of a business transaction:

```
@Transactional
public void transfer(Long fromId, Long toId, BigDecimal amount) {
    debitAccount(fromId, amount);
    creditAccount(toId, amount);
    recordTransfer(fromId, toId, amount);
}
```

Conceptually, the execution is:

```
Begin transaction
  ↓
Debit sender
  ↓
Credit receiver
  ↓
Record transfer
  ↓
Commit transaction
```

If an eligible failure occurs before successful completion:

```
Begin transaction
  ↓
Perform database operations
  ↓
Exception
  ↓
Rollback transaction
```

The transaction normally begins before the annotated method executes. It commits after the method completes successfully or rolls back when a qualifying exception leaves the method.

5. Begin, Commit and Rollback

5.1 Begin

BEGIN starts a database transaction.

At the JDBC level, manual transaction control usually begins by disabling autocommit:

```
connection.setAutoCommit(false);
```

With autocommit disabled, individual SQL statements are not committed automatically. The application must explicitly commit or roll back the transaction.

5.2 Commit

Commit means:

| Accept the transaction's changes as its final result.

At the JDBC level:

```
connection.commit();
```

After a successful commit, the changes become durable according to the guarantees of the database.

5.3 Rollback

Rollback means:

| Discard the changes made by the current uncommitted transaction.

At the JDBC level:

```
connection.rollback();
```

Rollback does not mean manually executing another **UPDATE** statement to restore every previous value. The database maintains the information required to undo or discard the uncommitted work.

6. Transaction Layers: JDBC, JPA, Hibernate and Spring

JDBC, JPA, Hibernate and Spring transactions are often presented as four different transaction systems. In a typical Spring application, they are better understood as different APIs and abstraction layers that control or coordinate the same underlying database transaction.

```
Service Method
  ↓
Spring Transaction Management
  ↓
JPA EntityManager / Hibernate Session
  ↓
JDBC Connection
  ↓
Database Transaction
```

6.1 JDBC Transaction

JDBC provides direct, low-level transaction control through a database connection:

```
Connection connection = dataSource.getConnection();

try {
    connection.setAutoCommit(false);

    // Execute SQL operations

    connection.commit();
} catch (Exception exception) {
    connection.rollback();
    throw exception;
} finally {
    connection.close();
}
```


The developer is responsible for:

- Obtaining the connection.
- Disabling autocommit.
- Committing on success.
- Rolling back on failure.
- Closing the connection safely.
- Preserving the original exception correctly.

6.2 JPA Transaction

JPA provides transaction control through its persistence API, commonly using `EntityManager.getTransaction()` in resource-local environments:

```
EntityManager transaction = entityManager.getTransaction();

try {
    transaction.begin();

    // Persistence operations

    transaction.commit();
} catch (Exception exception) {
    transaction.rollback();
    throw exception;
}
```

JPA hides direct JDBC connection handling, but the application still contains transaction boilerplate when transactions are managed manually.

6.3 Hibernate Transaction

Hibernate exposes its own native transaction API through `Session` and `Transaction`:

```
Transaction transaction = session.beginTransaction();

try {
    // Hibernate operations
    transaction.commit();
} catch (Exception exception) {
    transaction.rollback();
    throw exception;
}
```

When Hibernate is used as the JPA provider, JPA operations are ultimately implemented by Hibernate, which then interacts with JDBC and the database.

6.4 Spring-Managed Transaction

Spring removes transaction-management boilerplate from business methods.

```
@Transactional
public void transfer(Long fromId, Long toId, BigDecimal amount) {
    debitAccount(fromId, amount);
    creditAccount(toId, amount);
    recordTransfer(fromId, toId, amount);
}
```

The business method contains only business logic. Spring handles the surrounding transaction workflow:

```
Open or obtain required resources
Begin transaction
Execute business method
Commit on success
Roll back on eligible failure
Release resources
```

Spring does not replace the database transaction. It manages the lower-level transaction API on behalf of the application.

7. Spring's `@Transactional`

In a Spring Boot application, the commonly used annotation is:

```
import org.springframework.transaction.annotation.Transactional;
```

It tells Spring that the method or class should execute within a Spring-managed transaction.

7.1 Method-Level Transaction

Annotate a specific method when only selected operations require transaction management:

```
@Service
public class AccountService {

    @Transactional
    public void transfer(Long fromId, Long toId, BigDecimal amount) {
        // Business operations
    }
}
```

Here, the transaction applies to `transfer()`.

7.2 Class-Level Transaction

Annotate the class when all relevant public methods should use transactional behavior:

```
@Service
@Transactional
public class AccountService {

    public void transfer(...) {
```

```

    }

    public void closeAccount(...) {
    }

    public AccountResponse getAccount(...) {
    }
}

```

The class-level annotation provides a default transactional configuration for the class. A method-level annotation can provide more specific configuration for an individual method.

Transaction boundaries are generally placed in the **service layer** because a service method represents a complete business use case and may coordinate multiple repository operations.

8. Complete Mental Model

When Spring executes a transactional service method, the process can be understood as follows:

```

Caller invokes service method
↓
Spring starts or joins a transaction
↓
Repository operations use resources participating in that transaction
↓
Method completes successfully → Commit

or

Method fails with a rollback-eligible exception → Rollback

```

The key idea is:

`@Transactional` defines the boundary of a business operation, while Spring coordinates the underlying persistence and database transaction APIs needed to execute that operation safely.

9. Key Takeaways

- A transaction groups operations that must behave as one logical unit.
- A business operation may require one SQL statement or many SQL statements.
- Atomicity ensures that partial work is not committed.
- Consistency protects valid rules and invariants.
- Isolation controls interaction between concurrent transactions.
- Durability preserves committed results after failures.
- Begin, commit and rollback define the transaction lifecycle.
- JDBC, JPA and Hibernate expose different transaction APIs over the persistence stack.
- Spring coordinates transaction management and removes repetitive boilerplate.
- `@Transactional` is most commonly applied at the service layer around a complete business operation.